Threaded Code in logicforth

This document is a primer on how interpreters for threaded-code languages organize their inner dispatch loop, with logicforth as the worked example. By the end you should understand:

- What "threaded code" means, why it has nothing to do with concurrency, and where it sits between an AST walker and a JIT compiler
- How *indirect threading* works in general terms, and how logicforth implements it
- How *direct threading* works in general terms, and what logicforth's implementation would look like if it adopted it instead
- What the difference costs at runtime, and what it implies for the surrounding interpreter — the compiler that emits compiled code, the GC's walk over compiled bodies, the image save/load format, the trampoline `execute_cfa` uses to call colon-def words from C

logicforth's current implementation lives in `src/c/core.c` (look for `run_inner`, `execute_cfa`, `docol`, `dovar`, `dosym`, and `mark_body`) and the per-call emission code in `run_outer`. The differences between the two organizations are mechanically small but conceptually shift how every compiled word body is encoded, so a careful walk through the model is worth the time.

Part 1: The inner loop

Take a simple Forth-style program: `1 2 3 4 + + +`. It pushes four numbers, applies `+` three times, and leaves their sum on the stack.

Inside the interpreter, this program is a sequence of seven virtual operations: push 1, push 2, push 3, push 4, add, add, add. To run the program, the interpreter executes them in order. For each one, it figures out which piece of C code implements that operation, then runs that code.

That requires a small loop:

```
while (running) {
    /* fetch the next operation */
    /* find the C function that implements it */
    /* call the function */
}
```

The loop runs once per virtual operation. In a program of any size — a few hundred iterations of an inner Forth loop, millions of iterations across a benchmark — the loop iterates many millions or billions of times. Whatever happens inside it is on the critical path of every program the interpreter ever runs.

How that loop is organized — what it fetches per iteration, what it dereferences before knowing the handler, what call mechanism it uses to invoke that handler — determines the per-operation overhead of the whole language. A loop that does two memory loads per iteration pays twice the load cost of one that does a single load. A loop that calls through a function pointer pays the indirect-call cost. Each choice is small in isolation and significant in aggregate, because of the multiplier.

The cost is also bounded below by something. Even with the best possible organization of the loop, each virtual operation has to be fetched, dispatched, and have its handler invoked. The handler itself then does the actual work — popping operands, computing, pushing results. The handler's work is unavoidable. What's avoidable, or at least minimizable, is the cost of getting from "next instruction in the compiled stream" to "the first machine instruction of the handler that runs it." That cost is what threaded-code organizations differ on.

This document is about two of the most common organizations: *indirect threading* and *direct threading*. They differ in how each virtual operation in the compiled stream resolves to the C function that implements it, and the difference appears directly in the per-iteration work the loop does. The exposition walks through each in general terms, then shows what each looks like — or would look like — inside logicforth.

Part 2: What "threaded" means here

The word "thread" in *threaded code* has nothing to do with concurrency. It comes from textile imagery: a thread connecting beads on a string. A "threaded program" is one whose executable form is a sequence of small addresses — each one a bead — that the interpreter chases like sliding along a string.

This usage predates the modern concurrency sense by decades. It came out of the early Forth implementations of the 1960s and 70s, where machine memory was tiny and every byte mattered. A program encoded as a list of "addresses of routines" was both compact (each op is one cell) and fast to dispatch (no parsing, no integer-keyed lookup). The name stuck even after both of those original motivations became less pressing.

In modern parlance, here is the spectrum of how a program can be "interpreted":

- **AST walker.** Parse the source into a tree of nodes; walk the tree, dispatching on each node's type. Simple to implement; very slow. Every step traverses pointers between scattered heap-allocated nodes, fighting the cache; every node-type test is a switch or virtual call. Many small, one-shot scripting languages start here.
- **Bytecode interpreter.** Compile the program down to a flat array of small integer opcodes. The dispatch loop reads the next opcode and switches on it. CPython works this way. Cache-friendly (the opcode array is dense), but every dispatch has to look up the handler for an integer opcode — usually via a switch statement the compiler turns into a jump table.
- **Threaded code.** Instead of integers that index into a handler table, each op in the compiled stream is *itself* the dispatch target — an address (or an index that the dispatcher dereferences once). The integer opcode → handler lookup step that a bytecode interpreter pays for every op is gone. This is what logicforth uses.
- **JIT compilation.** Generate actual machine code on the fly. No interpretation loop at all — the CPU executes the generated code directly. Fastest, but requires emitting native instructions, managing executable memory, and dealing with code patching as new words are defined. PyPy, V8, the JVM, .NET.

Threaded code sits between bytecode and JIT. It gets some of JIT's speed (no integer dispatch) without any of JIT's complexity (no native code generation). It pays a little more than JIT per op (still has a

dispatch loop) but a lot less than bytecode (no integer-to-handler lookup per op).

logicforth's choice of threaded code rather than bytecode is largely historical — Forth-family languages have used threaded code since the beginning, and the interpreter was written in that style. But it's also the right choice for this language: the dictionary, the inner interpreter, and the relationship between compile-time `emit` and run-time dispatch all fall out naturally from the threaded-code model. Bytecode would have required a separate opcode space, a translation layer between the dictionary and the executable stream, and a handler table on the side. Threaded code lets the dictionary itself be the executable.

Within threaded code, two main flavors exist: **indirect** and **direct**. The difference is how each op in the compiled stream points at its handler. logicforth uses the indirect flavor today; this document is about moving to the direct flavor.

Part 3: A useful starting point — subroutine threading

Before we look at the two flavors that interpreters actually use, it helps to consider an extreme case: what if we didn't interpret at all, and the compiled program *was* native code?

That's roughly what a JIT compiler produces. In a Forth-flavored language, the most direct version is called *subroutine threading* (STC): each op becomes a literal CPU `CALL` instruction pointing at its handler, and the "compiled body" of a word is a sequence of those CALL instructions emitted straight into memory marked executable. The CPU's own program counter walks from CALL to CALL; there's no dispatch loop at all. Each handler ends in `RET`, which returns to the CALL site, which is immediately followed by the next CALL.

In this scheme, "dispatch" is what the CPU already does for free between instructions. There's no interpreter overhead — the cost per op is the CALL + RET round-trip, which is a couple of nanoseconds on modern hardware.

So why doesn't every interpreter do this? Three reasons:

1. **You have to generate native code.** This means knowing the target architecture's calling convention, emitting the right bytes for each CALL, managing executable memory pages (mprotect on Linux, VirtualProtect on Windows). Per-architecture work. Per-OS work. Unportable.

2. **Code isn't relocatable.** The CALL targets are absolute (or PC-relative) addresses. If you move a handler, every compiled body referencing it breaks. Image save/load becomes hard.

3. **You can't easily inspect or hook the dispatch.** Profiling, tracing, debugging — all require sitting in the middle of dispatch, which the CPU doesn't let you do in any portable way without injecting trampolines.

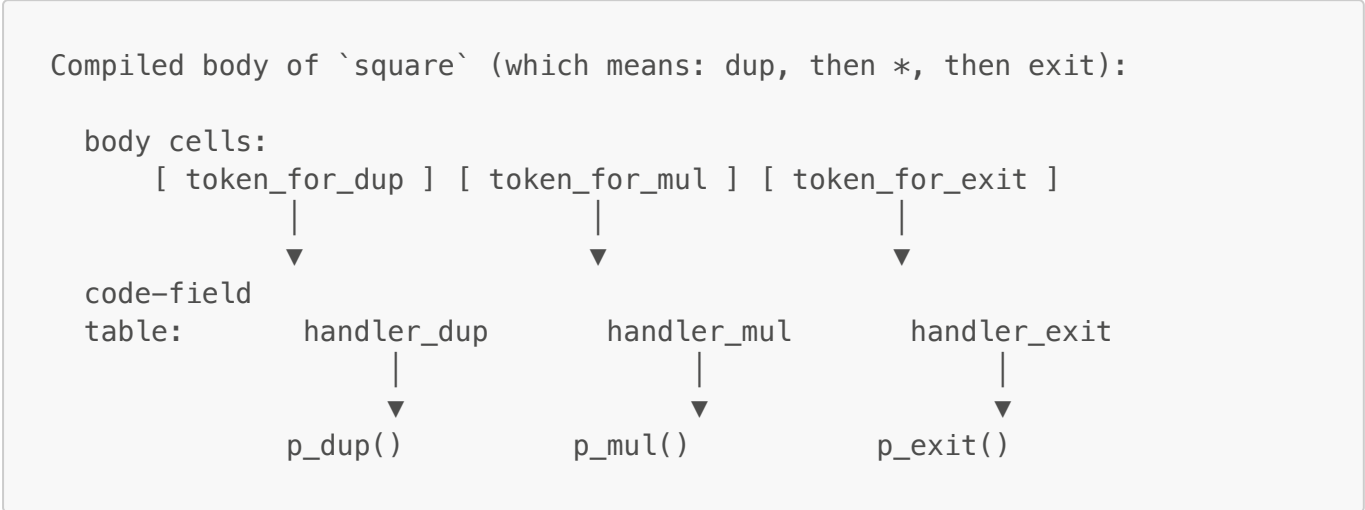
Interpreted threading (indirect or direct) keeps the dispatch in software, in one place we control. The cost: a few nanoseconds per op vs. STC's fewer-than-one. The benefits: portability, relocatability, inspectability.

STC is mentioned here purely as a conceptual anchor. The dispatch mechanisms we're about to compare both put a loop in software, with the op stream as data and the loop reading "the next op" each iteration.

Part 4: Indirect threading — the model

In *indirect-threaded code* (ITC), the compiled body of a word is an array of dispatch tokens. Each token doesn't directly identify a handler — it identifies a *code field*, which in turn identifies the handler.

The conceptual layout:



Each `token_for_X` is a small integer (or pointer) identifying X's code field. The code field is a memory cell holding the handler — the actual C function pointer that implements `X`. Dispatch is two reads:

```
read body cell  → token
read code-field at token → handler
call handler
```

The "indirect" part of the name is the second read. The body doesn't point straight at the handler; it points at a code-field cell that, in turn, holds the handler. Two memory accesses to get from "the next op" to "the function I'm about to call."

Why this design? Two practical reasons that mattered enormously when ITC was invented and still matter some today:

- **Different word kinds, same body shape.** In Forth, a colon-defined word and a variable are very different things — one has a body to walk, the other is a memory cell whose contents you push. But from the *caller's* point of view, both look identical: emit one token referring to the target. The handler at the code field disambiguates: colon-defs have `docol` as their handler, variables have `dovar`, symbols have `dosym`. The body cells of the caller don't care which.
- **Stable cross-process identifiers.** A token can be a small integer index, not a memory pointer. Indices don't change when the program is saved to disk and loaded later (ASLR randomizes pointers, but indices into a stable array are reproducible). Image save/load is trivial — just dump the cells; on load, the same indices still resolve to the same code fields.

Both of these matter to logicforth. The first is the reason `: foo bar ;` and `variable foo` can coexist in the same vocabulary with the same calling convention. The second is the reason `save-image` works as a flat copy of the dictionary today.

The cost is the second read. Every dispatched op pays for it.

Part 5: Direct threading — the model

In *direct-threaded code* (DTC), the compiled body of a word is an array of *handler addresses directly*. The token-to-code-field indirection is gone:

```
Compiled body of `square`:

  body cells:
      [ &p_dup ] [ &p_mul ] [ &p_exit ]
           │          │          │
           ▼          ▼          ▼
        p_dup()    p_mul()   p_exit()
```

Dispatch is one read:

```
read body cell  → handler
call handler
```

That's the entire change in one sentence: replace the index that points at a code-field cell with the function pointer that the code-field cell holds.

The benefit is immediate: one less memory load per dispatched op. On a loop where 74% of CPU is the dispatch step, removing half the loads from that step is the largest single available win.

The cost is in three places, all of them manageable:

1. **Body cells now contain absolute function pointers.** Pointers aren't stable across runs — every process has a different base address for its text segment under ASLR. Image save/load has to translate handler pointers to stable handler-ids on save and back on load. We'll cover this in detail in Part 14.

2. **Different word kinds need to thread their identity through.** When a body cell says "call `dovar`," the handler needs to know *which* variable's storage to fetch from. In ITC the token told it (the token *was* the variable's code-field index, and `dovar` read its operands from there). In DTC we lose that information when we collapse the indirection — unless we add it back as an extra operand cell. This is what Part 11 is about.

3. **Anywhere that inspects body cells** (the GC's `mark_body`, the image writer) has to switch from "compare against cfa-index" to "compare against handler-pointer." Mechanical update, several places.

The change is small in lines of code and large in mental model: every compiled body is now a stream of native function pointers, not a stream of indices. The next several parts walk through how this lands in logicforth specifically.

Part 6: Why indirect threading isn't just obsolete

It would be easy to read the above and conclude that ITC is a relic — a 1960s design that survives only by inertia. That's not quite right. ITC's extra indirection bought real things, and it's worth being explicit about what we're giving up by moving to DTC.

- **Code-field uniformity at the call site.** In ITC, every body cell is the same shape: one token. Whether the target is a primitive, a colon def, a variable, or a symbol, the body cell looks identical. DTC breaks this — primitive references are one cell, but colon-def/variable/symbol references need two (handler pointer + target identity). Bodies become shape-heterogeneous. Compilers and inspectors have to know per-op shape. (Today's `mark_body` already does this for `(literal)` and `(branch)` and the locals ops, so it's not a new burden — just an expanded one.)

- **Image portability.** ITC tokens are stable across runs; pointers aren't. Today, `save-image` dumps the dict cells raw and the loader reads them raw. After DTC, we need a per-cell encoding step on save ("translate this handler pointer to its handler-id enum") and a per-cell decoding step on load ("translate this id back to the live handler pointer at this run's addresses"). The image format gets a version bump and gets meaningfully more code. We'll show what this looks like in Part 14.

- **Variable handlers come for free.** A Forth dialect that wants to add a new word kind — say, a "constant" that pushes a fixed value when invoked — in ITC just allocates a new handler, assigns it to the new word's code field, and the dispatcher handles it identically. In DTC, the body-cell-emission code has to learn about the new kind too, because the body cell layout depends on whether the handler needs an operand. Modest cost.

- **Smaller dispatch token on banked-memory architectures.** A token in ITC can be a 32-bit (or smaller) index even on a 64-bit machine; a pointer in DTC has to be the architecture's full pointer width. On systems where this matters (embedded), ITC is more compact. Not a consideration for logicforth — it targets modern 64-bit systems where cells are already 64-bit ints.

- **Debuggability.** A token is a small integer that prints as a reasonable number; you can correlate "token 47" with "the 47th word in the dictionary." A handler pointer is an opaque 64-bit address. Useful debugging output gets harder. (The `find_word_by_handler_ptr` reverse lookup is straightforward but slower than reverse-lookup-by-index, and needs to walk the dictionary chain.)

So ITC's advantages are real. DTC's only advantage is speed — but on a profile where 74% of CPU sits in the dispatch loop, that advantage is decisive. The benefits ITC bought are either (a) no longer urgent in 2026 (cross-platform compactness) or (b) still achievable with a small amount of additional machinery (image format, GC dictionary walk). The benefit we'd gain — measurable wall-time speedup, plus the foundation for further dispatch optimizations like computed-goto — outweighs them.

This is the design tradeoff we're making. It's worth naming it explicitly so the next person to read this code understands that ITC wasn't a mistake; it was the right tradeoff for the original constraints, and DTC is the right tradeoff for the current ones.

Part 7: How logicforth implements indirect threading

Now we move from the abstract model to the actual code. The dictionary layout, the cells in it, the dispatch loop that walks them.

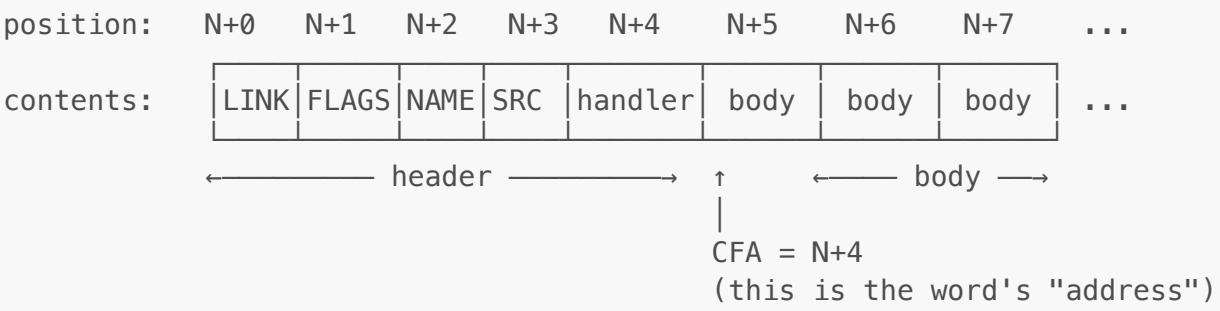
The dictionary is one big `cell[]` array — where `cell` is `typedef int64_t cell;`:

```
typedef struct Vocabulary {
    cell *dict;                 // the dictionary: one giant int64 array
    int dict_cap;
    int here;                   // next free position in dict
    int latest_cfa;             // head of the dictionary chain
    /* ... pools for names, source, symbols ... */
} Vocabulary;
```

`here` is the bump-allocator high-water mark — the index of the next free cell. `latest_cfa` is the position of the most-recently-defined word's *code field address* (its CFA). Every word's metadata header chains to the previous word via its LINK cell at offset `cfa - 4`, so the entire dictionary is a singly-linked list walkable from `latest_cfa`.

A word's anatomy

Each word in the dictionary occupies a sequence of cells:



The header is 4 cells: LINK, FLAGS, NAME, SRC (name-pool offset and source-pool offset). After the header sits the *code field*, a single cell whose contents is a C function pointer — the word's *handler*. The handler decides what happens when this word is called.

Following the code field is the body — a sequence of additional cells. For primitive words, there's no body; the code field's handler does the work itself and that's all there is to the word. For other word kinds, the body contains either the word's data (for variables and symbols) or a compiled stream of operations (for colon-defined words).

The word's *CFA* is the position of the code-field cell. By convention, "the CFA of foo" is the integer `N+4` in the diagram above. That number identifies the word — it's what `find()` returns, what `'` (tick) captures into an execution token, and what compiled body cells reference.

The `WORD_*` macros in `logicforth.h` access the metadata by offset from the CFA:

```
#define WORD_LINK(v, cfa)   ((v)->dict[(cfa) - 4])
#define WORD_FLAGS(v, cfa)  ((v)->dict[(cfa) - 3])
```

```
#define WORD_NAME(v, cfa)   ((v)->dict[(cfa) - 2])
#define WORD_SOURCE(v, cfa) ((v)->dict[(cfa) - 1])
```

So the CFA is a pivot: header metadata is at negative offsets, the handler is at offset 0, the body starts at offset 1.

Four handler kinds

Each word has one of four handler kinds in its code field cell:

- **A primitive handler.** A C function like `p_add`, `p_dup`, `p_frame_get`. The body is empty (or, more precisely, doesn't exist — the next thing in the dictionary is the next word's header). The handler does the work itself, pulling arguments from the data stack and pushing results back.
- **`docol`.** The handler for colon-defined words. The body is a stream of CFA references (the indirect-threaded code we're discussing) that call other words in sequence. `docol` knows how to begin walking that body.
- **`dovar`.** The handler for variables. The body is two cells: a Val tag and a Val data. When called, `dovar` reads those two cells and pushes the corresponding Val onto the data stack. When `to` writes to the variable, it overwrites those same two cells.
- **`dosym`.** The handler for symbol words declared with the `symbol` defining word. The body is one cell: the symbol-pool offset of the interned symbol name. When called, `dosym` reads that offset and pushes a `T_SYMBOL` Val.

`define_primitive` shows the simplest of these — building a word whose entire definition is a handler pointer in the code field:

```
int define_primitive(Interpreter *interp, const char *name,
                     cfa_handler handler, int immediate) {
    int cfa = create_header(interp, name, immediate);
    emit(interp, (cell)handler);     // CFA cell directly holds the handler
    return cfa;
}
```

The cast `(cell)handler` puts a C function pointer into an `int64_t` cell. On 64-bit platforms (the only ones logicforth targets), function pointers fit in 64 bits and survive the round-trip. The code-field cell *is* the handler.

So already, the code-field cell holds the handler directly — that's not the indirection ITC adds. The indirection is at the next level up: in the compiled body of *other* words, when they reference this primitive, they don't store the handler pointer. They store the CFA — an index into the dictionary — and the dispatcher reads `dict[cfa]` to recover the handler.

Emit and compile

When `: foo bar baz ;` is being compiled, `run_outer` parses each token in turn. When it sees `bar`, it calls `find("bar")` to get bar's CFA, then emits that CFA into the dictionary:

```
int cf = find(interp, tok);
if (cf) {
    if (interp->compiling && !WORD_IS_IMMEDIATE(interp->vocab, cf))
        emit(interp, (cell)cf);          // <-- this is the indirect bit
    else
        execute_cfa(interp, cf);
    continue;
}
```

The body cell holds `cf` — the CFA of `bar` as a `cell` (int64). At runtime, when foo runs and the dispatcher reaches this body cell, it'll read `cf`, then read `dict[cf]` to get bar's handler, then call it.

That's the two-read structure of ITC made concrete in logicforth.

Part 8: The dispatch loop in detail

The inner interpreter is `run_inner` in `core.c`. Stripped of the exception/unwinding handling (covered in `docs/continuations.md`), the hot loop is:

```
void run_inner(Interpreter *interp) {
    while (interp->running && !interp->error_flag) {
        int cfa_index       = (int)interp->vocab->dict[interp->ip++];   // read 1
        cfa_handler handler = (cfa_handler)interp->vocab->dict[cfa_index];
// read 2
        handler(interp, &interp->vocab->dict[cfa_index]);
    }
}
```

Read it carefully — this is the loop that took 74% of CPU.

1. **First read:** `dict[ip++]` fetches the next body cell at the instruction pointer, then advances `ip`. The body cell holds a CFA — a dictionary index identifying which word to call next.
2. **Second read:** `dict[cfa_index]` looks up the handler at that CFA. For a primitive, this is `&p_add` or `&p_dup` etc. For a colon-defined target, it's `&docol`. For a variable, `&dovar`. For a symbol, `&dosym`.
3. **Indirect call:** `handler(interp, ...)` invokes the handler. The second argument is the address of the CFA cell itself, so the handler can find its body if it needs to (docol does; dovar reads cfa[1] and cfa[2]; dosym reads cfa[1]; primitives mostly ignore it).

The two reads are the whole story of "indirect threading." One to get the op token from the body. One to dereference the token into the handler. The call happens after.

The `cfa_handler` signature

Every handler shares this signature:

```
typedef void (*cfa_handler)(Interpreter *interp, cell *cfa);
```

The `cfa` argument is the address of the called word's CFA cell. For `p_literal`, this means `cfa[1]` is the literal's tag and `cfa[2]` is its data. For `docol`, the implementation is:

```
void docol(Interpreter *interp, cell *cfa) {
    rpush(interp, make_addr(interp->ip));
    interp->ip = (int)(cfa - interp->vocab->dict) + 1;
}
```

`docol` saves the current instruction pointer onto the return stack (so the caller knows where to come back to), then computes `cfa - dict_base + 1` — the position one past the CFA cell, which is the body's first cell — and sets `ip` to that. The next iteration of `run_inner` will dispatch the first cell of the body.

For `dovar`:

```
void dovar(Interpreter *interp, cell *cfa) {
    Val v;
    v.tag = (Tag)cfa[1];
    v.data = cfa[2];
    push(interp, v);
}
```

Reads two cells past the CFA — that's where the variable's storage lives — constructs a Val from them, pushes it. No body walking, no `ip` manipulation; the variable's "execution" is just "pack two cells into a Val and push it."

For `dosym`:

```
void dosym(Interpreter *interp, cell *cfa) {
    int symbol_offset = (int)cfa[1];
    push(interp, make_symbol(symbol_offset));
}
```

Reads one cell past the CFA — the symbol-pool offset — wraps it as a `T_SYMBOL` Val, pushes.

The pattern: each handler reads what it needs from `cfa[1..N]`, using its known body layout. The dispatcher is generic; the handler is specific.

The trampoline trick in `execute_cfa`

`run_inner` is the in-loop dispatcher; `execute_cfa` is the function C code calls when it wants to invoke a word from outside the loop. The two paths handle primitives identically — just call the handler:

```
void execute_cfa(Interpreter *interp, int cfa) {
    cfa_handler handler = (cfa_handler)interp->vocab->dict[cfa];
    if (handler != &docol) {
        handler(interp, &interp->vocab->dict[cfa]);
        return;
    }
    /* docol case below */
}
```

For a primitive: read the handler, call it with `&dict[cfa]`, return. One line of real work.

But for a docol-handled word, we can't just call docol directly. docol sets `ip` to the body and returns — but if we're outside `run_inner`, nothing will dispatch the body. We need to invoke the inner loop on the body, then return when the body finishes.

The trick is a tiny *trampoline* — a fixed two-cell region at the start of the dictionary used solely for this purpose:

```
#define TRAMPOLINE_SLOT  0
#define DICT_RESERVED    2

interp->vocab->dict[TRAMPOLINE_SLOT]     = (cell)cfa;
interp->vocab->dict[TRAMPOLINE_SLOT + 1] = (cell)interp->vocab->stop_cfa;
interp->ip = TRAMPOLINE_SLOT;
interp->running = 1;
run_inner(interp);
```

The dictionary's first two cells (positions 0 and 1) are reserved at startup, never used for real word storage. To call a docol-handled word from C:

1. Write the target's CFA into `dict[TRAMPOLINE_SLOT]`. This makes the trampoline slot look exactly like a body cell that calls our target.
2. Write `stop_cfa` (the CFA of the `(stop)` primitive, which sets `running = 0`) into `dict[TRAMPOLINE_SLOT + 1]`. This is the "what to do when the target returns."
3. Set `ip = TRAMPOLINE_SLOT`. The dispatcher will start reading from the trampoline.
4. Run `run_inner`. The dispatcher reads `dict[0]` (our target's CFA), reads `dict[cfa]` (docol), calls docol. docol saves the current ip (= 1, the slot of `stop_cfa`) onto the return stack and jumps into the target's body. The body executes; on its final `exit`, the saved ip (= 1) is popped, the dispatcher reads `dict[1]` (stop_cfa), reads `dict[stop_cfa]` (= `p_stop`), calls it. `p_stop` sets `running = 0`. The loop exits.

5. Back in `execute_cfa`, we restore the saved `ip` from before the call.

The trampoline is a tiny piece of dictionary cleverness that lets the out-of-loop call mechanism reuse the in-loop dispatcher. Two cells of reserved dictionary space (`DICT_RESERVED = 2`), constant overhead.

Part 9: Body shapes for every word kind

Now we have the dispatcher and the per-kind handlers. To finish the picture of ITC in logicforth, we need to enumerate exactly what shows up in a compiled body — because everything that walks bodies (the GC, the forthcoming DTC rewrite of the image format, the conceptual model of "what dispatch reads") needs to know.

Primitive references

A body cell referencing a primitive is one cell: the target's CFA. The dispatcher reads the CFA, dereferences it to get the primitive's handler, calls it. No operands; the handler pulls arguments from the data stack.

Example: `+` in a body emits one cell, `cfa_of_+`.

Colon-def references

A body cell referencing a colon-defined word is one cell: that word's CFA. Identical shape to a primitive reference at the call site — the distinction is in the handler. The dispatcher reads the CFA, dereferences it, finds `docol`, calls it. docol then begins walking the target's body.

Example: `square` in a body emits one cell, `cfa_of_square`.

Variable references

A body cell that *reads* a variable also emits one cell: the variable's CFA. The dispatcher reads it, dereferences to get `dovar`, calls `dovar`, which reads the variable's tag and data from cells past the CFA and pushes the corresponding Val.

Example: `count` (referring to a variable named `count`) in a body emits one cell, `cfa_of_count`. Same shape as a primitive or colon reference; again the handler differentiates.

Symbol references

A body cell that pushes a symbol declared via `symbol :foo` emits one cell: the symbol-word's CFA. Dispatcher → `dosym` → push the symbol.

Literal Vals (compile-time-known constants)

When `: foo 42 ;` compiles, the literal `42` becomes a three-cell sequence: the CFA of `(literal)` (the primitive that pushes a Val from inline operands), then a `Tag` cell, then a data cell:

```
void emit_val_literal(Interpreter *interp, Val value) {
    emit(interp, (cell)interp->vocab->literal_cfa);
    emit(interp, (cell)value.tag);
```

```
    emit(interp, value.data);
}
```

At runtime, the dispatcher reads `(literal)`'s CFA, dereferences to `p_literal`, calls it. `p_literal` reads two more cells from `ip` (the tag and data), constructs a Val, pushes it, and `ip` ends up advanced past all three cells.

```
void p_literal(Interpreter *interp, cell *cfa) {
    Val literal;
    literal.tag = (Tag)interp->vocab->dict[interp->ip++];
    literal.data = interp->vocab->dict[interp->ip++];
    push(interp, literal);
}
```

This is *the* pattern for any op with inline operands: the handler reads its operands from `dict[ip++]`, advancing `ip` to skip past them.

String literals (via `(dostr)`)

When `: greet "hello" ;` compiles, the string literal emits a two-cell sequence: the CFA of `(dostr)` and a handle. `p_dostr` reads the handle, interpolates the template if it has placeholders, pushes the resulting string. Two cells total per string literal.

Branches

`if`/`then`/`else`/`begin`/`until`/`again` all compile to one of two branch primitives — `(branch)` (unconditional) or `(0branch)` (branch if top-of-stack is zero) — followed by a one-cell offset:

```
[cfa_of_(0branch)] [offset] ...
```

`p_branch` and `p_0branch` read the offset cell and adjust `ip` by it.

Locals

The locals machinery introduces five ops with inline operands:

- `(enter-locals) <n>` — two cells. Pushes a locals frame of N slots.
- `(leave-locals) <n>` — two cells. Pops N slots and the saved local_base.
- `(local@) <depth> <slot>` — three cells. Read a local.
- `(local!) <depth> <slot>` — three cells. Write a local.
- `(to-var) <var-cfa>` — two cells. Store TOS to a global variable.

Each handler reads its operands from `dict[ip++]` past its own CFA, then does its work.

The complete catalogue

So body cells fall into three buckets:

1. **Dispatch cells.** One cell, holding a CFA. The dispatcher reads it, looks up the handler, calls it. Every body op starts with one of these.
2. **Operand cells.** Cells immediately following a dispatch cell that the handler reads. Their meaning is op-specific: a tag, a data field, a branch offset, a depth, a slot, a CFA-as-operand. The dispatcher never tries to interpret them as dispatch tokens.
3. **The `exit` cell.** At the end of every colon-def body sits the CFA of `(exit)`, which pops the return stack and resumes the caller. Just another dispatch cell, but worth naming because every body ends with one.

This three-bucket structure is what the GC's `mark_body` walks (it must know how many operand cells follow each dispatch cell, to avoid reading operands as dispatch tokens) and what the image-save format encodes (the encoding shape depends on the bucket).

Part 10: Direct threading — the redesign

With the existing ITC model now fully laid out, we can specify the direct-threaded version concretely.

The change is: **dispatch cells in compiled bodies now hold handler pointers directly, instead of CFAs that the dispatcher dereferences to find handlers.**

Operand cells are unchanged. The header is unchanged. The code field (the cell at the CFA position) is unchanged — it still holds the word's handler, which is what `find()` returns and what `execute_cfa` uses to identify the kind of word.

What changes is the body's encoding of "call this other word."

The new dispatch loop

```
void run_inner(Interpreter *interp) {
    while (interp->running && !interp->error_flag) {
        cfa_handler handler = (cfa_handler)interp->vocab->dict[interp-
>ip++];
        handler(interp, &interp->vocab->dict[interp->ip]);
    }
}
```

One read instead of two. The body cell *is* the handler pointer; the dispatcher invokes it directly. The second argument is `&dict[ip]` — the address of the cell immediately after the handler, where any operands live.

Side-by-side: ITC vs DTC dispatch

ITC, with comments showing what each cell holds:

```
Body cell at ip:      cfa_index    ——— one int64 holding an index into
dict[]
```

```
Dispatcher does:      read body cell → get cfa_index    [read 1]
                      read dict[cfa_index] → get handler  [read 2]
                      call handler(&dict[cfa_index])
```

DTC:

```
Body cell at ip:      handler_ptr  ——— one int64 holding a function pointer
Dispatcher does:      read body cell → get handler_ptr   [read 1]
                      call handler(&dict[ip])
```

One read. The function pointer is the body cell.

The new compile-time emission

In `run_outer` today, when the parser sees a known word name during compilation, it does:

```
emit(interp, (cell)cf);    // emit the target's CFA index
```

After DTC, it becomes:

```
emit_call(interp, cf);     // emit the right cells for this target's
handler kind
```

Where `emit_call` does the right thing based on the target's handler — a single cell for primitives, two cells for handlers that need to thread their target's identity through.

Part 11: The two-cell call site

Here's the one design wrinkle. In ITC, when a body cell holds the CFA of a variable, the dispatcher dereferences the CFA, finds `dovar`, and calls `dovar` with `&dict[cfa]` — and *that pointer* tells `dovar` where the variable's data lives. The CFA serves dual duty: it identifies the handler (via `dict[cfa]`) and it identifies the operand storage (via `dict[cfa+1]` and `dict[cfa+2]`).

After DTC, the body cell holds just the handler pointer (`&dovar`). The "which variable?" information is gone. If `dovar` is called now, it has no way to find which variable's data it's supposed to push.

The fix is a small one: any handler that needs to know "which target am I working on?" gets that target's CFA as an operand cell, immediately following the handler pointer. So a body that today is one cell (`cfa_of_var`) becomes two cells (`&dovar`, `cfa_of_var`).

Three handlers need this in logicforth:

- **`docol`** (the handler for colon-defined words) — needs to know the target's body start, so it can jump there.

- `dovar` (the handler for variables) — needs to know the variable's CFA, so it can read tag/data from CFA+1/CFA+2.
- `dosym` (the handler for symbol words) — needs to know the symbol word's CFA, so it can read the symbol-pool offset from CFA+1.

For each of these, the body cell expands from one cell to two: handler pointer + target CFA. The handler reads the operand cell, advances `ip` past it, then does its work using the target CFA.

The new docol

```
void docol(Interpreter *interp, cell *next) {
    int target_cfa = (int)next[0];
    interp->ip++;                                  /* skip past target_cfa
operand */
    rpush(interp, make_addr(interp->ip));
    interp->ip = target_cfa + 1;                   /* jump to body */
}
```

`next[0]` is the cell immediately after the handler pointer — the target CFA. Advance `ip` past it (so when this docol returns control to the dispatcher, the dispatcher reads the cell *after* the operand, not the operand itself). Push the post-operand `ip` onto the return stack (this is where `(exit)` will pop back to). Then set `ip` to `target_cfa + 1`, the first cell of the target's body.

The new dovar

```
void dovar(Interpreter *interp, cell *next) {
    int var_cfa = (int)next[0];
    interp->ip++;                                  /* skip past var_cfa
operand */
    Val v;
    v.tag = (Tag)interp->vocab->dict[var_cfa + 1];
    v.data = interp->vocab->dict[var_cfa + 2];
    push(interp, v);
}
```

Read the operand to find which variable's storage to fetch from, advance `ip`, fetch tag and data, push the Val.

The new dosym

```
void dosym(Interpreter *interp, cell *next) {
    int sym_cfa = (int)next[0];
    interp->ip++;
    int symbol_offset = (int)interp->vocab->dict[sym_cfa + 1];
    push(interp, make_symbol(symbol_offset));
}
```

Same shape. Read operand, advance, fetch from CFA-relative offset, push.

`emit_call`

The compile-time helper that emits the right cells for a given target:

```
void emit_call(Interpreter *interp, int target_cfa) {
    cfa_handler h = (cfa_handler)interp->vocab->dict[target_cfa];
    emit(interp, (cell)h);                          /* handler pointer */
    if (h == docol || h == dovar || h == dosym) {
        emit(interp, (cell)target_cfa);             /* target CFA as operand
*/
    }
    /* primitive handlers don't need a target — one cell is enough */
}
```

Called from `run_outer` (when the parser sees a known word during compilation), from `p_tick` (when `'` produces an xt), and anywhere else that today emits a CFA reference into a body.

What about primitives with their own inline operands?

`p_literal`, `p_branch`, `p_0branch`, `p_dostr`, `p_to_var`, `p_enter_locals`, `p_leave_locals`, `p_local_fetch`, `p_local_store` — these primitives already read inline operand cells immediately after their own CFA. Nothing about that changes in DTC. After the dispatcher reads the handler pointer and calls the handler with `&dict[ip]`, the handler reads operands from `dict[ip++]` as it does today.

So the operand-cell mechanism is unified: handlers that take operands read them from `dict[ip]` and advance `ip` past them. Whether the operand is a "tag" (for `p_literal`) or a "target CFA" (for the new `dovar`), it's just the next cell.

Why this is clean

The dispatcher stays dumb. It dispatches one handler per loop iteration and passes it the address of the cell that follows. Each handler knows its own operand shape and consumes the operands it needs by advancing `ip` itself.

The only asymmetry in the new emission code is: which handlers expect a "target CFA" operand? Currently three (`docol`, `dovar`, `dosym`). If a fourth kind is ever added — say a "constant" handler `docon` that holds a literal in its body — it joins the list. `emit_call` becomes a function of one switch over handler kind, mechanical to extend.

Part 12: A worked example — `: caller square 1 + ;`

Suppose we have:

```
: square dup * ;            \ a colon def
variable count              \ a variable
: caller square 1 + count ;
```

Three words to compile. Let's lay out the dictionary in both schemes and trace a dispatch.

Dictionary layout — ITC (current)

Pretend our dictionary starts at position 100 (skipping pre-loaded words for brevity). `square`, `count`, and `caller` get defined in order:

```
Pos    Cell                   Notes
100    link to prev word      \ square's header begins
101    flags = 0
102    name pool offset
103    source pool offset
104    &docol                 \ square's CFA — handler is docol
105    cfa_of_dup             \ body of square: dup ...
106    cfa_of_mul             \                 * ...
107    cfa_of_exit            \                 ; (implicit exit)
108    link to square's cfa   \ count's header
109    flags = 0
110    name pool offset
111    source pool offset
112    &dovar                 \ count's CFA — handler is dovar
113    T_FLOAT (Tag)          \ count's data: tag cell
114    0.0 bits               \               data cell
115    link to count's cfa    \ caller's header
116    flags
117    name pool offset
118    source pool offset
119    &docol                 \ caller's CFA — handler is docol
120    cfa_of_square (=104)   \ body of caller: square ...
121    cfa_of_(literal)       \                 (literal)
122    T_FLOAT                \                   tag
123    1.0 bits               \                   data
124    cfa_of_+               \                 + ...
125    cfa_of_count (=112)    \                 count ...
126    cfa_of_exit            \                 ; (implicit exit)
```

The body of `caller` (positions 120–126) is seven cells: five dispatch cells (square, literal, +, count, exit) plus two operand cells (tag and data for the literal).

Dictionary layout — DTC (after the change)

```
Pos    Cell                    Notes
100    link to prev word       \ square's header — unchanged
101    flags
102    name pool offset
103    source pool offset
104    &docol                  \ square's CFA — unchanged
105    &p_dup                  \ body of square: dup
106    &p_mul                  \                 *
107    &p_exit                 \                 ;
108    link                    \ count's header
109    flags
110    name pool offset
111    source pool offset
112    &dovar                  \ count's CFA — unchanged
113    T_FLOAT                 \              data cells unchanged
114    0.0 bits
115    link                    \ caller's header
116    flags
117    name pool offset
118    source pool offset
119    &docol                  \ caller's CFA
120    &docol                  \ body of caller: square's call site
121    104 (= cfa_of_square)   \                    operand: square's CFA
122    &p_literal              \                 (literal)
123    T_FLOAT                 \                   tag
124    1.0 bits                \                   data
125    &p_add                  \                 +
126    &dovar                  \                 count's read site
127    112 (= cfa_of_count)    \                   operand: count's CFA
128    &p_exit                 \                 ;
```

The body of `caller` (120–128) is nine cells: seven dispatch+operand groups. Two of those (square and count) are two-cell pairs because their handlers are `docol` and `dovar`. Everything else is one cell per op.

Body size grew from 7 to 9 cells — two more, one for each "anchored" reference. For caller-of-caller patterns (deeply nested colon defs), this adds up; for primitive-heavy bodies, it's negligible.

Dispatch trace: caller invoked once

Say we're at `ip = 120` and `running = 1`. The dispatcher runs:

ITC trace (reading caller's body at position 120):

```
ip=120: read dict[120] = cfa_of_square (= 104)    [read 1]
        read dict[104] = &docol                    [read 2]
        ip is now 121
        call docol(&dict[104])
        docol does: rpush(ip=121); ip = 105

ip=105: read dict[105] = cfa_of_dup                [read 1]
```

```
          read dict[cfa_of_dup] = &p_dup              [read 2]
          ip = 106
          call p_dup

  ip=106: read dict[106] = cfa_of_mul                 [read 1]
          read dict[cfa_of_mul] = &p_mul              [read 2]
          ip = 107
          call p_mul

  ip=107: read dict[107] = cfa_of_exit                [read 1]
          read dict[cfa_of_exit] = &p_exit            [read 2]
          ip = 108
          call p_exit
          p_exit does: ip = rpop().data = 121

  ip=121: read dict[121] = cfa_of_(literal)           [read 1]
          read dict[cfa_of_(literal)] = &p_literal    [read 2]
          ip = 122
          call p_literal(&dict[cfa_of_(literal)])
          p_literal reads dict[122] (tag), dict[123] (data), pushes 1.0
          ip is now 124

  ip=124: read dict[124] = cfa_of_+                   [read 1]
          read dict[cfa_of_+] = &p_add                [read 2]
          ip = 125
          call p_add

  ip=125: read dict[125] = cfa_of_count (= 112)       [read 1]
          read dict[112] = &dovar                     [read 2]
          ip = 126
          call dovar(&dict[112])
          dovar reads dict[113] (tag), dict[114] (data), pushes the Val

  ip=126: read dict[126] = cfa_of_exit                [read 1]
          read dict[cfa_of_exit] = &p_exit            [read 2]
          ip = 127
          call p_exit
          p_exit pops the caller's return address (whatever invoked caller)
```

Total dispatch reads to run caller's body once (not counting reads inside the called words): **14 reads** (7 dispatch cells × 2 reads each).

DTC trace (reading caller's body at position 120):

```
  ip=120: read dict[120] = &docol                     [read 1]
          ip = 121
          call docol(&dict[121])
          docol does: target_cfa = 104; ip++ (= 122);
                      rpush(ip=122); ip = 104+1 = 105

  ip=105: read dict[105] = &p_dup                     [read 1]
          ip = 106
```

```
          call p_dup

  ip=106: read dict[106] = &p_mul                    [read 1]
          ip = 107
          call p_mul

  ip=107: read dict[107] = &p_exit                   [read 1]
          ip = 108
          call p_exit
          ip = rpop().data = 122

  ip=122: read dict[122] = &p_literal                [read 1]
          ip = 123
          call p_literal(&dict[123])
          reads tag and data, pushes 1.0, ip = 125

  ip=125: read dict[125] = &p_add                    [read 1]
          ip = 126
          call p_add

  ip=126: read dict[126] = &dovar                    [read 1]
          ip = 127
          call dovar(&dict[127])
          dovar does: var_cfa = 112; ip++ (= 128);
                      reads dict[113], dict[114], pushes Val

  ip=128: read dict[128] = &p_exit                   [read 1]
          ip = 129
          call p_exit
          pops caller's return
```

Total dispatch reads: **7 reads** (one per dispatch cell, no double-read).

ITC: 14 reads. DTC: 7 reads. Each read is a load from cache (L1 in the hot loop, since dict is contiguous). At ~1 ns per L1 load, that's ~7 ns saved per body — and `caller` runs once per call. If `caller` is in a loop running 10M times, the saving is ~70 ms per benchmark run.

This is what the 10-15% projection comes from. The savings are asymptotic in proportion to dispatch-bound time.

Part 13: What the change touches

Outside the dispatch loop, the change reaches the following surfaces:

1. `emit_call` replaces direct emission of CFA cells

In `run_outer` and `p_tick`, the line that today reads:

```
emit(interp, (cell)cf);
```

becomes:

```
emit_call(interp, cf);
```

And `emit_call` is a new C function (~10 lines) that handles the one-cell vs two-cell distinction.

2. `docol`, `dovar`, `dosym` are rewritten

Each gains the small operand-reading prologue shown in Part 11. The post- prologue body is unchanged. Maybe 4 lines added to each, 12 total.

3. The dispatch loop in `run_inner` collapses

Two dict reads become one. The handler call uses `&dict[ip]` instead of `&dict[cfa_index]`. Net change: one fewer line, one fewer read.

4. `execute_cfa` updates

The non-docol path can still do `handler(interp, &dict[cfa])` for primitives that ignore the `cfa *` argument (most do). But `dovar` and `dosym` in DTC expect their first cell to be the *target CFA*, not the handler pointer. So `execute_cfa` either special-cases them (inline the push directly) or sets up a per-kind trampoline.

The cleanest version inlines:

```
void execute_cfa(Interpreter *interp, int cfa) {
    cfa_handler handler = (cfa_handler)interp->vocab->dict[cfa];
    if (handler == dovar) {
        Val v;
        v.tag  = (Tag)interp->vocab->dict[cfa + 1];
        v.data = interp->vocab->dict[cfa + 2];
        push(interp, v);
        return;
    }
    if (handler == dosym) {
        push(interp, make_symbol((int)interp->vocab->dict[cfa + 1]));
        return;
    }
    if (handler == docol) {
        /* trampoline — see below */
        ...
    }
    /* primitive — direct call, cfa* argument is unused */
    handler(interp, &interp->vocab->dict[cfa]);
}
```

5. `TRAMPOLINE_SLOT` widens to three cells

The current trampoline is two cells: target CFA + stop CFA. In DTC, to invoke a docol-handled word the trampoline becomes three cells: docol pointer + target CFA + stop handler pointer:

```
dict[TRAMPOLINE_SLOT]     = (cell)docol;
dict[TRAMPOLINE_SLOT + 1] = (cell)target_cfa;
dict[TRAMPOLINE_SLOT + 2] = (cell)p_stop;
interp->ip = TRAMPOLINE_SLOT;
run_inner(interp);
```

The dispatcher reads `docol` from slot 0, advances ip to 1, calls docol with `&dict[1]`. docol reads target_cfa from `dict[1]`, advances ip to 2, pushes 2 as the return address, jumps to the target body. The body runs; on exit, ip pops back to 2. Dispatcher reads `p_stop` from `dict[2]`, calls it. Done.

`DICT_RESERVED` becomes 3 instead of 2. Trivial change.

6. `mark_body` comparisons switch from CFA-indices to handler pointers

Today `mark_body` identifies operand-bearing ops by comparing dict cells against `literal_cfa`, `dostr_cfa`, `branch_cfa`, etc.:

```
if (ref == (cell)interp->vocab->literal_cfa && cursor + 2 < body_end) {
    /* this is a (literal) op — operands are tag, data */
    ...
}
```

In DTC, the dispatch cell is the handler pointer:

```
if (ref == (cell)(uintptr_t)&p_literal && cursor + 2 < body_end) {
    /* this is a (literal) op — operands are tag, data */
    ...
}
```

The logic — "how many operand cells follow each dispatch cell" — is identical. Only the comparison value changes. All the `literal_cfa`, `dostr_cfa`, `branch_cfa`, etc. fields in `Vocabulary` can be replaced by direct references to the handler functions, or kept as cached values of the handler pointers.

Plus the new wrinkle: `docol`, `dovar`, `dosym` references each have one operand cell (the target CFA). `mark_body` needs to know that those handlers come with a one-cell operand and skip it. So `mark_body` grows three more branches.

The operand cell after `docol` is a CFA — an integer index into `dict[]`. It's not a Val; it doesn't need marking. `mark_body` just skips it. Similarly for `dovar` and `dosym`.

7. `save-image` / `load-image` gain a handler-id translation layer

This is the biggest user-facing change. Today the image writer dumps the dict cells verbatim — they're all int64s and they're all meaningful as indices (which are stable across runs). The reader reads them back verbatim.

After DTC, dispatch cells in bodies hold function pointers. Function pointer addresses are randomized by ASLR — they differ from one process launch to the next, and certainly from one binary to another. Saving them verbatim and loading them in a different process means loading garbage.

The image writer needs to know, for each cell, what its semantic role is — is this a dispatch cell (needs handler-id encoding) or an operand cell (write verbatim)? That's exactly the same per-cell knowledge that `mark_body` already has. Both pieces of code walk bodies the same way, identifying dispatch cells by their handler-pointer values and skipping the right number of operand cells.

The image format gains a small handler-id table at the top: a list of every handler the binary uses, assigned a stable small integer. On save, each dispatch cell is replaced by its handler-id. On load, the loader maps each handler-id back to the live handler pointer in the current process and writes it into the dict.

The existing image format already has a tiny version of this — the `HANDLER_DOCOL` / `HANDLER_DOVAR` / `HANDLER_DOSYM` enum is used to record which kind of handler is at each user-defined word's CFA cell (because there are only a handful of possible values there). DTC generalizes the same idea to *every* dispatch cell, not just CFA cells. The enum widens to cover every primitive handler.

The image format gets a version bump (the existing one is 1; we'd become 2). Old images written by ITC binaries can't be loaded by DTC binaries directly without a one-time migration pass — but in practice we re-run `save-image` from the current binary, so this is mostly a backwards-incompatibility note rather than a real burden.

8. `forget`'s body walk — no change needed

`forget` rewinds `here`, `latest_cfa`, and the pool watermarks. It doesn't currently walk bodies, so nothing breaks.

Part 14: The image format in more detail

Let's spell out what the new image format looks like, since it's the largest concrete change in the codebase.

Today's image format roughly:

```
Magic: "LF4I"
Version: 1
Section: user_dict_cells      (raw int64s — body cells, CFA cells, header
cells)
Section: handler-id table     (cfa → HANDLER_DOCOL/DOVAR/DOSYM)
Section: name_pool / source_pool / symbol_pool
Section: objects[]
Section: data stack
```

The `handler-id table` exists only to record which CFA cells in the user-defined-words region hold which handler — because those handler pointers vary per binary and need to be re-resolved on load. There are only three handlers it covers (`docol`, `dovar`, `dosym`), and only one cell per user-defined word (the CFA cell).

After DTC, every dispatch cell in every body holds a handler pointer that needs re-resolution. The handler-id space expands from 3 to ~80 (every primitive). The save path becomes:

For each body cell, classify it as either:

- **Dispatch cell**: encode as `(HANDLER_TAG, handler_id)` — two values written, marking it as needing re-resolution on load.
- **Operand cell**: encode as `(OPERAND_TAG, raw_value)` — written verbatim, no resolution needed.

The classification uses the same per-op shape table `mark_body` uses. For each known op, after writing the dispatch cell with `HANDLER_TAG`, write the following N operand cells with `OPERAND_TAG`. Then advance to the next op.

On load, the reader:

- For each `HANDLER_TAG` cell, looks up the handler-id in the live binary's table and writes the resolved handler pointer.
- For each `OPERAND_TAG` cell, writes the raw value through.

The format grows from "one int64 per cell" to "two int64s per cell" (tag

- value). Image files roughly double in size. That's the cost. For context: today's image files are small (the entire vocab is maybe a few hundred KB), so even 2× is fine.

A more compact alternative — bitfields where each cell gets one bit indicating dispatch-vs-operand — would halve the size. The simple two-int64-per-cell format is clearer and the size is not a real constraint. We start with the simple version.

The version field in the header rises to 2. Images written by old binaries are flagged as incompatible at load time, with a clear error message pointing at the migration path (re-run the user's `.l4` source files in the new binary, then `save-image` again).

Part 15: Expected performance impact

Direct projections from the profile data:

- **Phase 1** (hand-rolled `begin/until` loop): currently 47 ms. ~74% of CPU is in the dispatch loop; halving the read count in that loop should land roughly a 10-15% wall-time improvement, so 40-42 ms.
- **Phase 5** (`map` + `reduce` over a range): currently 21 ms; expect 18 ms. Same dispatch-bound profile.
- **Phase 2** (`range → map → filter → reduce`): currently 17 ms; expect 15 ms. Similar.
- **Phase 4** (frame build + walk): currently 6 ms; expect 5 ms. Dispatch is a smaller fraction here because frame ops do more work per dispatch.
- **Phase 6** (deep nested frames): currently 9 ms; expect 8 ms.

- **Phase 3** (DGEMM): currently 0.12 ms. *Unaffected* — DGEMM is a single C-kernel call, no dispatch loop involvement.

Total benchmark time projected: from ~99 ms to ~87 ms (~12% wall-time improvement). The bulk of the gain is in phases that spend most of their time dispatching virtual ops.

There's a secondary benefit not captured in the read-count analysis: branch prediction. The indirect call in the dispatch loop has a single call site that varies in target. Modern CPUs (M1/M2 ARM64, recent x86) predict indirect branches well by recording the most-recent target at each call site. With one fewer dependent load before the call (DTC vs ITC), the call target becomes available sooner, the predictor has less to guess about, and prediction accuracy improves.

The third benefit is cache pressure. One less L1 read per dispatch means slightly fewer L1 evictions of the body-cell array. For benchmark-scale code that fits in L1 entirely, this doesn't matter; for larger programs it can.

The projection numbers above are conservative — they assume only the read-count saving. Real measurements may come in a few percent better if branch prediction and cache effects compound.

Part 16: What comes after — computed-goto threading

DTC is the first step. The natural next step is *computed-goto threading* (also called *token threading* in some literature) — a technique where the inner loop is replaced by inline `goto` statements at the end of every primitive's body, dispatching directly to the next handler without returning to a central loop.

In C with GCC/Clang's labels-as-values extension:

```
void *table[] = { &&l_dup, &&l_mul, &&l_add, &&l_exit, ... };

l_dup:    /* p_dup body */
          goto *(void**)dict[ip++];

l_mul:    /* p_mul body */
          goto *(void**)dict[ip++];

/* etc */
```

Each handler dispatches to the next one inline. The branch predictor can now see the *specific transition* (dup → mul, mul → add, etc.) at each dispatch site, not just "an indirect jump." Hot transitions get predicted; cold ones don't. CPython adopted this in 3.11 and reported 15-20% speedup.

The catch: each handler can no longer be a separate C function. They all have to live as labels inside one giant function — typically the inner interpreter loop itself. logicforth's primitives are currently spread across `core.c`, `words.c`, `collections.c`, `matrix.c`, `functional.c` as separate `void p_X(...)` functions. Adopting computed-goto means consolidating them into one big switch-with-labels.

That's a bigger refactor than DTC. DTC keeps the existing function structure intact. Computed-goto would require rewriting the dispatch loop and inlining (or `goto`-ing into) every handler.

DTC is a prerequisite for computed-goto: you need body cells to hold direct dispatch targets, not indices that require a second read before the goto. DTC lays the foundation; computed-goto exploits it.

We don't have to do computed-goto. DTC alone provides a solid incremental win. Computed-goto is in the deferred-work pile.

Part 17: The big picture

In slogan form:

> **ITC: body cells are indices. Dispatch is two reads. DTC: body cells are pointers. Dispatch is one read.**

The change replaces one level of indirection (CFA-index → handler) with a direct pointer in the compiled body. The cost is image-format complexity (pointers aren't portable; we now translate them on save/load) and a slightly larger compiled body (colon-def, variable, and symbol references take two cells instead of one).

The benefit is wall-time speedup on dispatch-bound workloads (~10-15% per the projection) and the architectural foundation for the next-tier dispatch optimization (computed-goto threading).

The details that make it work:

- A small `emit_call` helper that knows to emit one cell for primitives, two cells for handlers that thread their target's identity through (currently `docol`, `dovar`, `dosym`).
- Updated `docol` / `dovar` / `dosym` handlers that read their target CFA from the operand cell, advance `ip` past it, then do the same work they do today.
- A one-line-shorter dispatch loop in `run_inner`.
- A widened trampoline (`DICT_RESERVED` from 2 to 3) so `execute_cfa` can invoke docol-handled words from C.
- Mechanical updates to `mark_body` and the image format to know about three new operand-bearing op shapes and to translate handler pointers through a stable handler-id table.
- A version bump on the image file format.

Total size of the change: a few hundred lines across the dispatch path, the compiler, the GC body walker, and the image format. The conceptual shift is "every body cell that today is an index becomes the value it indexed." Once that lands, the next idea (computed-goto) becomes addressable.

Part 18: Where to look in the source

Indirect-threaded (current):

- **`run_inner`** in `src/c/core.c` — the dispatch loop. The two-read pattern is `cfa_index = dict[ip++]; handler = dict[cfa_index];`.

- **`docol` / `dovar` / `dosym`** in `src/c/core.c` — the non-primitive handlers. Each reads its data via the `cfa *` argument.
- **`execute_cfa`** in `src/c/core.c` — the C-side entry point. The docol case uses the `TRAMPOLINE_SLOT` / `DICT_RESERVED` mechanism.
- **`define_primitive`** in `src/c/core.c` — the simplest defining word. Shows that CFA cells already hold handler pointers directly; the indirection is only in body cells of *other* words.
- **`run_outer`** in `src/c/core.c` — the parser/compiler. The line `emit(interp, (cell)cf);` in the "known word during compilation" branch is the source of every ITC dispatch cell in every body.
- **`emit_val_literal`** and `p_literal` in `src/c/core.c` — the prototype for "primitive with inline operands." Operand reading via `dict[ip++]` in the handler is the pattern other operand-bearing ops (branches, locals, `dostr`, `to-var`) all follow.
- **`mark_body`** in `src/c/core.c` — the dictionary walker. The per-op branches encode the same body-shape knowledge that DTC needs in `emit_call` and the image format.
- **`p_save_image` / `p_load_image`** in `src/c/core.c` — the current image format. The `HANDLER_DOCOL` / `HANDLER_DOVAR` / `HANDLER_DOSYM` enum is the tiny prototype that DTC's full handler-id table generalizes.

Direct-threaded (planned):

- `PLAN.md`, section "Interpreter performance — next up," has the short-form spec. This document is the long-form one.
- The change shows up in: `run_inner`, `docol`, `dovar`, `dosym`, `execute_cfa`, `mark_body`, `p_save_image`, `p_load_image`, and a new `emit_call` helper. The compile-time call site in `run_outer` (and `p_tick`) switches from `emit(...)` to `emit_call(...)`.
- The `Vocabulary` struct's `literal_cfa` / `dostr_cfa` / `branch_cfa` / etc. fields can either be kept (as cached handler-pointer values) or removed (replaced by direct references to the handler functions). Each approach is a style choice; either works.

For broader context:

- **`docs/gc.md`** — the GC primer. The `mark_body` walker that DTC also needs to update is documented there with the per-op branches that inform the DTC body-shape table.
- **`docs/continuations.md`** — the continuation primer. The dispatch loop's relationship to `run_inner`, `execute_cfa`, and the trampoline is covered there in the context of continuation capture; DTC's modifications preserve the same shape, just with one fewer read per dispatched op.